Part-of-Speech Tagging

How to tackle the task?

- Rule-based tagging
- HMM tagging
- Transformation-based tagging
- Maximum entropy tagging
- Using Neural Networks (LSTM, RNN, ….)
- ….

Transformation Based Tagging/ Brill Tagging

- Draws inspiration from both the rule-based and statistical taggers
- Like the rule-based taggers, TBL is based on rules that specify what tags should be assigned to what words
- But like the statistical taggers, TBL is a machine-learning technique, in which rules are automatically induced from the data
- it assumes a pre-tagged training corpus.

Transformation Based Tagging/ Brill Tagging

- How TBL rules are applied?

1. is expected to race tomorrow

2. the race for outer space

$$P(\text{NN}|\text{race}) = .98$$
$$P(\text{VB}|\text{race}) = .02$$

1. is/VBZ expected/VBN to/TO race/NN tomorrow/NN

2. the/DT race/NN for/IN outer/JJ space/NN

Transformation Rule

Change NN to VB when the previous tag is TO

expected/VBN to/TO race/NN ——->. expected/VBN to/TO race/VB

Transformation Based Tagging/ Brill Tagging

- How TBL rules are learned?

✓ *TBL algorithm has **three** major stages*

- ✤ *It first labels every word with its most likely tag.*
- ✤ *It then examines every possible transformation and selects the one that results in the most improved tagging.*
- ✤ *Finally, it then re-tags the data according to this rule.*
- ✤ *The last two stages are repeated until some stopping criterion is reached, such as insufficient improvement over the previous pass.*

Transformation Based Tagging/ Brill Tagging

- How TBL rules are learned?

✓ *TBL algorithm has **three** major stages*

- ✤ *It first labels every word with its most likely tag.*
- ✤ *It then examines every possible transformation and selects the one that results in the most improved tagging.*
- ✤ *Finally, it then re-tags the data according to this rule.*
- ✤ *The last two stages are repeated until some stopping criterion is reached, such as insufficient improvement over the previous pass.*

Transformation Based Tagging/ Brill Tagging

- How TBL rules are learned?

✓ *In principle the set of possible transformations is infinite*
✓ *But TBL needs to consider every possible transformation*
✓ *In order to pick the best one on each pass through the algorithm.*
✓ *Thus the algorithm needs a way to limit the set of transformations.*
✓ *This is done by designing a small set of templates (abstracted transformations).*

The preceding (following) word is tagged **z**.
The word two before (after) is tagged **z**.
One of the two preceding (following) words is tagged **z**.
One of the three preceding (following) words is tagged **z**.
The preceding word is tagged **z** and the following word is tagged **w**.
The preceding (following) word is tagged **z** and the word two before (after) is tagged **w**.

Transformation Based Tagging/ Brill Tagging

- How TBL rules are learned?

#	Change tags From	To	Condition	Example
1	NN	VB	Previous tag is TO	to/TO race/NN → VB
2	VBP	VB	One of the previous 3 tags is MD	might/MD vanish/VBP → VB
3	NN	VB	One of the previous 2 tags is MD	might/MD not reply/NN → VB
4	VB	NN	One of the previous 2 tags is DT	
5	VBD	VBN	One of the previous 3 tags is VBZ	

Syntax Analysis

What is Syntax?

- **Syntax** refers to the way words are arranged together
- **Syntax** captures structural relationships between words and phrases
- Constituents: Noun Phrase, Verb Phrase, Determiners….
- Grammars are used to describe the syntax of a language,
  - Context Free Grammars

Why is Syntax necessary?

- Natural language is **more than bag of words**
- We ignore meaning and simply look at syntax.

colourless green ideas sleep furiously

What is Parsing?

- An automatic analysis of a sentence with respect to its syntactic structure.
- The process of determining the grammatical structure with respect to a given grammar.
- Given a grammar, this means deriving a phrase structure tree assigned to the sentence by the grammar

Why can't we use FSAs?

- Language is left/right recursive:
  ✓ {tall, green, slimy, calm, …}* frog
  ✓ the house {with a roof, with a door, with a window, with a garden, …}*
  ✓ Can process these recursions with FSAs: ADJ* NN , NN (P DET NN)*

- But language has also center-embedded recursions:
  ✓ the door opens
  ✓ the door that uncle Henry repaired opens
  ✓ the door that uncle Henry who the dog bit repaired opens
  ✓ the door that uncle Henry who the dog that John owns bit repaired opens

- Center-embedding recursion is not regular

Context Free Grammar

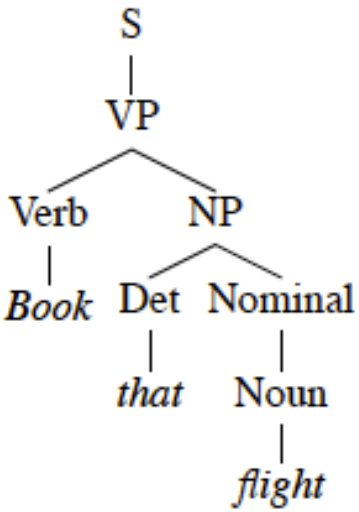
- Modeling via **Context Free Grammars (CFG)**

- G = (T, N, S, R)
  - T is set of terminals
  - N is set of nonterminals
  - S is the start symbol (one of the nonterminals)
  - R is set of production rules of the form X → ***Y***, where X is a nonterminal and ***Y*** is a sequence of terminals and nonterminals
- A grammar G generates a language L

Example Grammar

$S \rightarrow NP\ VP$	$Det \rightarrow that \mid this \mid a$
$S \rightarrow Aux\ NP\ VP$	$Noun \rightarrow book \mid flight \mid meal \mid money$
$S \rightarrow VP$	$Verb \rightarrow book \mid include \mid prefer$
$NP \rightarrow Pronoun$	$Pronoun \rightarrow I \mid she \mid me$
$NP \rightarrow Proper\text{-}Noun$	$Proper\text{-}Noun \rightarrow Houston \mid TWA$
$NP \rightarrow Det\ Nominal$	$Aux \rightarrow does$
$Nominal \rightarrow Noun$	$Preposition \rightarrow from \mid to \mid on \mid near \mid through$
$Nominal \rightarrow Nominal\ Noun$	
$Nominal \rightarrow Nominal\ PP$	
$VP \rightarrow Verb$	
$VP \rightarrow Verb\ NP$	
$VP \rightarrow Verb\ NP\ PP$	
$VP \rightarrow Verb\ PP$	
$VP \rightarrow VP\ PP$	
$PP \rightarrow Preposition\ NP$	

Parse Tree



Sentences are generated by recursively rewriting the start symbol using the productions until only terminal symbols remain

Types of Parsing

- Given a string of terminals and a CFG, determine if the string can be generated by the CFG.
  - ✓ return a parse tree for the string
  - ✓ return all possible parse trees for the string

. Top-down Parsing -
Start searching the space of derivations from the start symbol

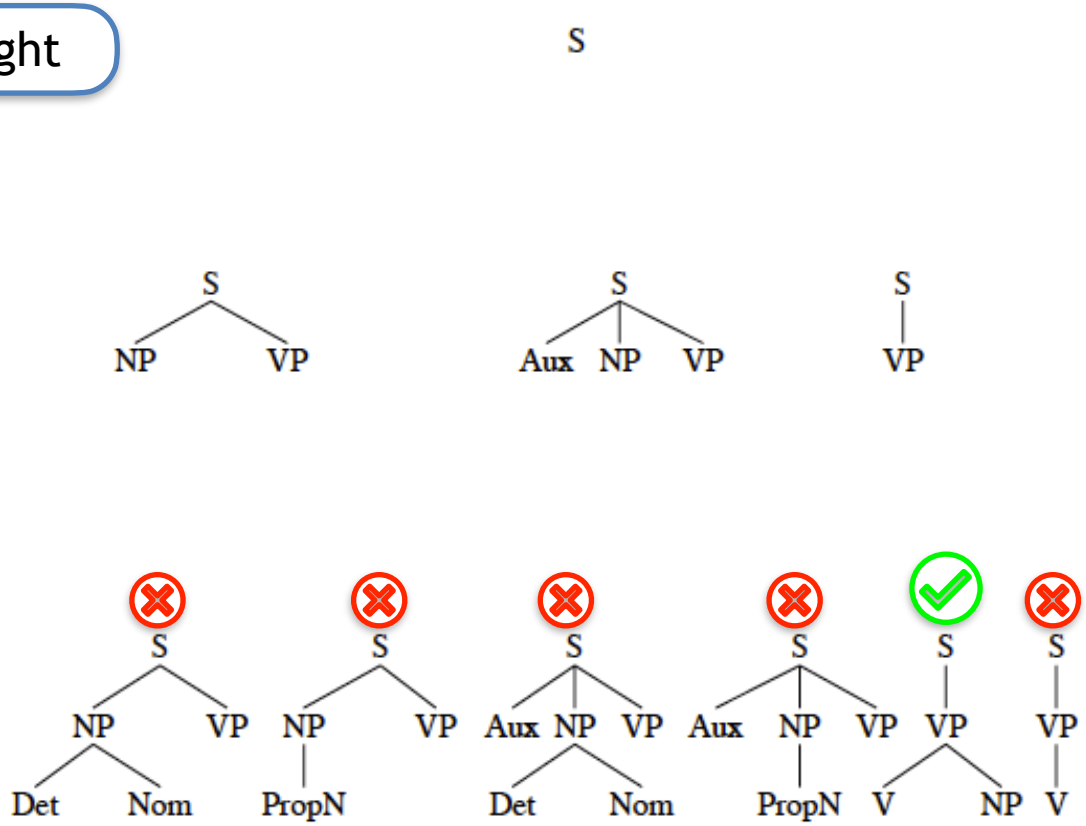
. Bottom-up Parsing -
Start search space of reverse derivations from the terminal symbols in the string

Top-Down Parsing

$S \rightarrow NP\ VP$
$S \rightarrow Aux\ NP\ VP$
$S \rightarrow VP$
$NP \rightarrow Pronoun$
$NP \rightarrow Proper\text{-}Noun$
$NP \rightarrow Det\ Nominal$
$Nominal \rightarrow Noun$
$Nominal \rightarrow Nominal\ Noun$
$Nominal \rightarrow Nominal\ PP$
$VP \rightarrow Verb$
$VP \rightarrow Verb\ NP$
$VP \rightarrow Verb\ NP\ PP$
$VP \rightarrow Verb\ PP$
$VP \rightarrow VP\ PP$
$PP \rightarrow Preposition\ NP$

$Det \rightarrow that \mid this \mid a$
$Noun \rightarrow book \mid flight \mid meal \mid money$
$Verb \rightarrow book \mid include \mid prefer$
$Pronoun \rightarrow I \mid she \mid me$
$Proper\text{-}Noun \rightarrow Houston \mid TWA$
$Aux \rightarrow does$
$Preposition \rightarrow from \mid to \mid on \mid near \mid through$

book that flight

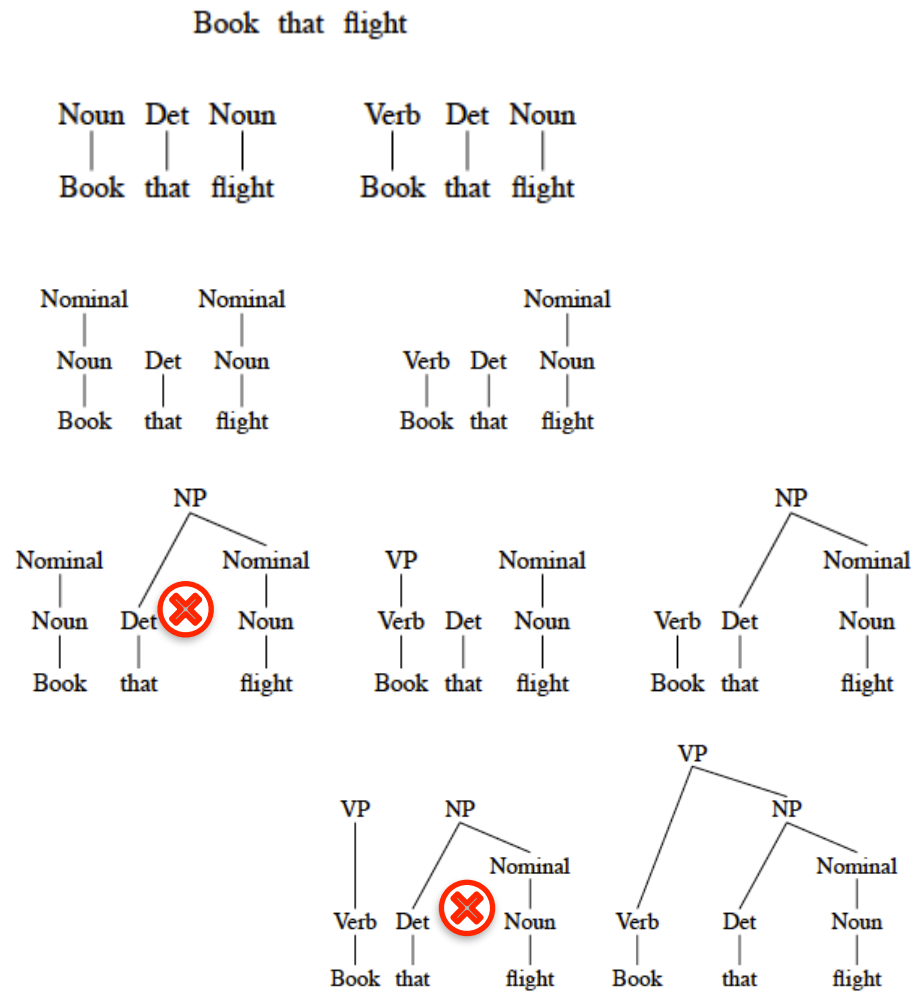


Bottom-up Parsing

$S \rightarrow NP\ VP$
$S \rightarrow Aux\ NP\ VP$
$S \rightarrow VP$
$NP \rightarrow Pronoun$
$NP \rightarrow Proper\text{-}Noun$
$NP \rightarrow Det\ Nominal$
$Nominal \rightarrow Noun$
$Nominal \rightarrow Nominal\ Noun$
$Nominal \rightarrow Nominal\ PP$
$VP \rightarrow Verb$
$VP \rightarrow Verb\ NP$
$VP \rightarrow Verb\ NP\ PP$
$VP \rightarrow Verb\ PP$
$VP \rightarrow VP\ PP$
$PP \rightarrow Preposition\ NP$

$Det \rightarrow that \mid this \mid a$
$Noun \rightarrow book \mid flight \mid meal \mid money$
$Verb \rightarrow book \mid include \mid prefer$
$Pronoun \rightarrow I \mid she \mid me$
$Proper\text{-}Noun \rightarrow Houston \mid TWA$
$Aux \rightarrow does$
$Preposition \rightarrow from \mid to \mid on \mid near \mid through$

book that flight



Top-down vs Bottom-up

- Top-down never explores options that will not lead to a **full parse**, but can explore many options that never connect to the **actual sentence**.
- Bottom-up never explores options that do not connect to the actual sentence but can explore options that can never lead to a full parse.
- Relative amounts of wasted search depend on how much the grammar branches in each direction.
- Complexity of naive implementation: Exponential due to branching

Parsing using Dynamic Programming

- To avoid extensive repeated work, must cache intermediate results, i.e. completed sub-phrases.

- Caching (memorizing) is critical to obtaining a polynomial time parsing algorithm for CFGs.

- Dynamic programming algorithms based on both top-down and bottom-up search can achieve $O(n^3)$ recognition time where n is the length of the input string.

Dynamic Programming Parsing Methods

- CYK (Cocke-Younger-Kasami, 1967) algorithm based on **bottom-up** parsing and requires first normalizing the grammar.

- Earley (1970) parser is based on **top-down** parsing, does not require normalizing the grammar, but is more complex.